Fiche sur le théorème maître

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

08 octobre 2024

Le théorème maître en algorithmique permet de déterminer la complexité temporelle asymptotique en $\Theta()$ de certains algorithmes dont la complexité temporelle est définie selon une formule de récurrence. Cette fiche se découpe en 4 parties :

- une première partie qui présente la formule de récurrence traitée par ce théorème et qui donne une intuition de comment interpréter le théorème maître,
- une deuxième partie qui énoncera le théorème maître dans sa forme classique présentée en cours,
- Une troisième partie qui présente une méthode pratique pour appliquer le théorème maître.
- Une dernière partie qui donne des rappels sur la manipulation des logarithmes et puissances, indispensables à la bonne compréhension de cette fiche et à la manipulation du théorème maître.

Le théorème maître n'est pas à connaître par coeur. En revanche, il est susceptible de faire l'objet d'une question au cours du DS, et l'inclure dans vos 5 pages de notes personnelles autorisées semble donc *plus que raisonnable*...

1 Formule de récurrence et compréhension du théorème

Le théorème maître permet de traiter des formules de récurrence de la forme

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

avec $a \geqslant 1$ et $b > 1$. Afin de mieux comprendre comment interpréter le théorème maître, on va développer cette expression, puis l'écrire sous la forme de la forme d'une série.

$$
\begin{aligned}
T(n) &= aT\left(\frac{n}{b}\right) + f(n) \\
&= a\left(aT\left(\frac{n}{b \times b}\right) + f\left(\frac{n}{b}\right)\right) + f(n) \\
&= a^2 T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= a^2\left(aT\left(\frac{n}{b^3}\right) + f\left(\frac{n}{b^2}\right)\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= a^3 T\left(\frac{n}{b^3}\right) + a^2 f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= \dots \\
&= a^{t-1} T\left(\frac{n}{b^{t-1}}\right) + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + \cdots + a^2 f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= a^{t-1}\left(aT\left(\frac{n}{b^t}\right) + f\left(\frac{n}{b^{t-1}}\right)\right) + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + \cdots + a^2 f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n)
\end{aligned}
$$

On fait, à partir d'ici, l'hypothèse que $n = b^t$, c'est-à-dire que n est une puissance de b. On supposera aussi que $T(1) = \Theta(1)$. On peut alors simplifier certains termes, et faire des regroupements :

$$
\begin{aligned}
T(n) &= a^{t-1}\left(aT\left(\frac{n}{b^t}\right) + f\left(\frac{n}{b^{t-1}}\right)\right) + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + \cdots + a^2 f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= a^t T\left(\frac{n}{b^t}\right) + a^{t-1} f\left(\frac{n}{b^{t-1}}\right) + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + \cdots + a^2 f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= a^t T\left(\frac{b^t}{b^t}\right) + \left(f(n) + af\left(\frac{n}{b}\right) + a^2 f\left(\frac{n}{b^2}\right) + \cdots + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + a^{t-1} f\left(\frac{n}{b^{t-1}}\right)\right) \\
&= a^t T(1) + \left(a^0 f\left(\frac{n}{b^0}\right) + a^1 f\left(\frac{n}{b^1}\right) + a^2 f\left(\frac{n}{b^2}\right) + \cdots + a^{t-2} f\left(\frac{n}{b^{t-2}}\right) + a^{t-1} f\left(\frac{n}{b^{t-1}}\right)\right) \\
&= a^t \Theta(1) + \sum_{j=0}^{t-1} a^j f\left(\frac{n}{b^j}\right) \\
&= \Theta\left(a^t\right) + \sum_{j=0}^{t-1} a^j f\left(\frac{n}{b^j}\right)
\end{aligned}
$$

On a fait plus tôt l'hypothèse que $n = b^t$, donc $\log_b(n) = \log_b(b^t) = t\log_b(b) \Leftrightarrow t = \log_b(n)$, car $\log_b(b) = 1$. Ainsi, on peut réécrire :

$$
T(n) = \Theta\left(a^t\right) + \sum_{j=0}^{t-1} a^j f\left(\frac{n}{b^j}\right) = \Theta\left(a^{\log_b(n)}\right) + \sum_{j=0}^{\log_b(n)-1} a^j f\left(\frac{n}{b^j}\right)
$$

Le terme $a^{\log_b(n)}$ cache enfait un polynôme. Pour s'en rendre compte, il faut se rappeler de la définition de la puissance par l'exponentielle : $x^y = \mathrm{e}^{y\ln(x)}$, avec x strictement positif ici. En appliquant cette formule à notre terme :

$$a^{\log_b(n)} = e^{\log_b(n)\times\ln(a)} = e^{\frac{\ln(a)}{\ln(b)}\times\ln(n)} = e^{\frac{\ln(n)}{\ln(a)}\times\ln(b)} = e^{\log_b(a)\times\ln(n)} = n^{\log_b(a)}$$

par définition de $\log_b(x) = \frac{\ln x}{\ln b}$, par commutativité du produit usuel, et par la formule pour x^y décrite plus haut. On peut donc utiliser cette notation dans $T(n)$:

$$T(n) = \Theta\left(a^{\log_b(n)}\right) + \sum_{j=0}^{\log_b(n)-1} a^j f\left(\frac{n}{b^j}\right) = \Theta\left(n^{\log_b(a)}\right) + \sum_{j=0}^{\log_b(n)-1} a^j f\left(\frac{n}{b^j}\right)$$

qui, finalement, en posant $c = \log_b(a)$, s'écrit :

$$T(n) = \Theta\left(n^c\right) + \sum_{j=0}^{\log_b(n)-1} a^j f\left(\frac{n}{b^j}\right)$$

$T(n)$ s'exprime donc maintenant selon la somme de deux nouveaux termes : $\Theta\left(n^c\right)$ qui en réalité est une famille de fonctions polynômiales de degré c, et la somme $\sum_{j=0}^{\log_b(n)-1} a^j f\left(\frac{n}{b^j}\right)$. De ce fait, si l'on s'intéresse à la complexité temporelle de $T(n)$ et en particulier à sa complexité asymptotique en $\Theta()$, il faut s'intéresser à la relation de dominance qui existe entre les deux termes qui le définissent. C'est là l'idée du théorème maître : **considérer trois cas différents qui statuent sur la relation de dominance entre les deux termes calculés**.

La preuve du théorème maître n'est pas donnée ici, car il n'est pas attendu de vous de la connaître ou de savoir la refaire. Si toutefois vous souhaitez aller plus loin, elle est décrite dans l'ouvrage de référence que constitue *Introduction to Algorithms - Fourth edition* de H. Cormen et cie.

2 Énoncé du théorème maître

Theorem 1 (Théorème maître)**.** *On considère la formule de récurrence suivante :*

$$\begin{cases} T(n) = aT\left(\frac{n}{b}\right) + f(n),\ n \geqslant 2 \\ T(1) = O(1) \end{cases}$$

avec $a \geqslant 1$, $b > 1$, $n = b^t$ et $c = \log_b(a)$. Alors, il est possible de donner la complexité asymptotique en $\Theta()$ de $T(n)$ dans trois cas :

Cas numéro 1. *Si $f(n) = O\left(n^{c'}\right)$, avec $c' < c$, alors $T(n) = \Theta\left(n^c\right)$.*

Cas numéro 2. *Si $f(n) = \Theta\left(n^c\right)$, alors $T(n) = \Theta\left(n^c \log(n)\right)$*

Cas numéro 3. *Si $f(n) = \Omega\left(n^{c'}\right)$, avec $c' > c$ **et** si $af\left(\frac{n}{b}\right) \leqslant \alpha f(n)$ pour un certain $\alpha \leqslant 1$ et pour $n \geqslant n_0$, alors $T(n) = \Theta\left(f(n)\right)$.*

Remarque importante : En fonction des valeurs de a et b et de la forme de $f(n)$, on ne vérifiera pas nécessairement un des trois cas ! Le théorème maître ne couvre qu'un nombre limité de formules de récurrence. Par exemple, toutes les fonctions $f(n) = \Theta\left(n^c \log(n)\right)$ ne satisferont aucun des trois cas présentés ici. Il existe d'autres travaux qui généralisent

cette formule à plus de cas, en particulier le théorème d'Akra-Bazzi quand on fait plusieurs appels récursifs de taille inégale. Cela étant dit, **toutes les formules de récurrences que vous recontrerez dans le cours, et dans les examens, entreront nécessairement dans l'un des trois cas présentés.** Ainsi, si vous concluez que votre formule de récurrence ne correspond à aucun des trois cas, c'est qu'il y a un problème quelque part...

3 Méthode pour appliquer le théorème maître

Je vous propose ici une méthode pratique afin de résoudre le plus efficacement possible le problème de la détermination de la complexité asymptotique de $T(n)$ à partir de sa formule de récurrence en passant par le théorème maître, et ce tout en minimisant le risque d'erreurs de calculs.

La méthode consiste à réaliser plusieurs étapes dans un ordre bien particulier, que voici :

1. Identifier les paramètres a et b du problème, et définir $f(n)$ par sa formule.
2. Calculer $c = \log_b(a)$.
3. Tester chaque cas du théorème maître, en respectant les points suivants :
   — **On traite les cas dans l'ordre suivant : deuxième cas, premier cas puis enfin troisième cas.**
   — **Lorsque l'un des cas est vérifié, il est inutile de poursuivre et on ne traite pas les cas restants.**
4. Une fois l'un des cas vérifié, conclure quant à la notation asymptotique en $\Theta()$ en utilisant l'énoncé du théorème maître.

Cette méthode n'est bien-sûr pas la seule méthode possible, mais les deux premières étapes vous forcent à ne pas vous lancer directement dans les calculs ou raisonnements et donc à bien identifier les éléments que vous devrez utiliser pour ces derniers ; et l'ordre de traitement des cas du théorème maître est choisi de sorte à minimiser les calculs ou raisonnements à faire.

Bonus

Cette section « bonus » décrit un certain nombre de formules et résultats notables utiles à la manipulation du théorème maître. Ce sont par ailleurs des résultats globalement utile dans les matières scientifiques (informatique, mathématiques, physique).

Dans toutes les formules qui suivent, $\ln(\cdot)$ représente le logarithme néperien, et $\log_b(\cdot)$ le logarithme en base b.

— $\log_b(x) = \frac{\ln(x)}{\ln(b)}$
— $\ln(x \times y) = \ln(x) + \ln(y)$, qui donne également $\log_b(x \times y) = \log_b(x) + \log_(y)$
— $\ln\left(\frac{x}{y}\right) = \ln(x) - \ln(y)$, qui donne également $\log_b\left(\frac{x}{y}\right) = \log_b(x) - \log_b(y)$
— $\ln\left(x^y\right) = y \times \ln(x)$, qui donne également $\log_b\left(x^y\right) = y \times \log_b(x)$
— $\log_b(1) = \frac{\ln 1}{\ln b} = \frac{0}{\ln b} = 0$
— $\log_b(b) = \frac{\ln b}{\ln b} = 1$

— $\ln(e^x) = x$ et $\log_b(b^x) = x$

La formule de passage entre la puissance et l'exponentielle est $x^y = e^{y \times \ln(x)}$, avec x strictement positif.